Week 1 Day-by-Day Study Planner: Foundations & Setup

15-Hour Action Plan: Computational Thinking, Visual Studio Code, GitHub Setup & Your First Interactive Program

Week 1 Mission & Daily 2-Hour Study Block Formula

Your sole focus in Week 1 is removing setup friction and learning how programmers break problems down into inputs, algorithms, and outputs. For Monday through Friday, divide your **2-hour daily session** into three blocks: **00:00–00:35** (Active concept study with notes in VS Code), **00:35–01:50** (Hands-on building & exercises), and **01:50–02:00** (Save work to GitHub + write a 3-line daily log).

Day-by-Day Schedule & Step-by-Step Tasks (Monday – Sunday)

Day & Time	Session Focus & Links	Step-by-Step Tasks to Complete	Done When (Output)
Monday 2.0 Hours	**Developer Environment & GitHub Setup** VS Code & GitHub Skills	Install Visual Studio Code and review the Getting Started Guide. Create a local `software-engineering-journey` folder, open a GitHub account, and finish Introduction to GitHub.	VS Code is installed locally and your first repository is live on your GitHub profile.
Tuesday 2.0 Hours	**Binary, ASCII & Algorithmic Thinking** CS50x Lecture 0 Notes	Watch the first 60 minutes of Harvard CS50x Week 0 (Binary, ASCII, RGB, Algorithms). In VS Code, create `week1-notes.md` and write step-by-step pseudocode (with `if/else` and loops) for 3 tasks.	`week1-notes.md` saved in VS Code explaining binary and linear vs. binary search.
Wednesday 2.0 Hours	**Functions, Loops, Events & Scratch** MIT Scratch & CS50x Lecture 0	Watch the second half of CS50x Lecture 0 covering variables, boolean expressions, loops, and functions. Open the Scratch Getting Started Editor and rebuild the 3 lecture demos yourself.	3 small experimental scripts built in Scratch using loops, variables, and custom blocks.
Thursday 2.0 Hours	**Web Architecture & Odin Orientation** Odin Foundations & MDN Web	Create your account on The Odin Project Foundations and complete How This Course Will Work. Study MDN: How the Web Works (DNS, HTTP, packets).	Written summary in `week1-notes.md` tracing what happens when you visit a URL.
Friday 2.0 Hours	**Project Architecture & Spec Design** CS50x Problem Set 0 Spec	Read all requirements on CS50x Problem Set 0. Sketch an original interactive mini-game or story on paper first, then build the player controls and state variables in Scratch Editor.	Core player/sprite movement and state reset logic working in your Scratch project.
Saturday 3.5 Hours	**Deep Project Build & Debugging** CS50x Problem Set 0	Implement all required graduation criteria: at least 2 sprites, 3+ scripts total, 1 conditional (`if/else`), 1 loop, 1 variable, and 1 custom block (function) with an input. Play-test and debug edge cases.	Complete, bug-free interactive project meeting 100% of CS50x Problem Set 0 specs.
Sunday 1.5 Hours	**Submission, Git Upload & Week 2 Prep** CS50x Week 1 (C)	Submit your finished project via CS50x Problem Set 0. Upload your `.sb3` file and `week1-notes.md` to your GitHub repo with a clean `README.md`, and preview CS50x Week 1.	Problem Set 0 submitted, Week 1 GitHub repo updated, and Week 2 schedule locked in.

Week 1 Graduation Checklist (Verify Before Moving to Week 2)

- **[ ] Environment Ready:** Visual Studio Code is installed and you can create, edit, and save Markdown (`.md`) files using keyboard shortcuts.
- **[ ] GitHub Active:** You have a public GitHub profile, completed Introduction to GitHub, and uploaded your Week 1 study notes.
- **[ ] Core CS Concepts Mastered:** You can explain in your own words how binary represents numbers/text, why binary search is faster than linear search, and what a function parameter does.
- **[ ] First Project Shipped:** Your original interactive project is submitted to CS50x Problem Set 0 with at least one custom parameterized block, loop, variable, and conditional branch.